

Pupilometer Test

Documentation for Fove Unity Visual Stimulation and Eye Tracking Project

Background & Research

A Unity-based application that delivers dynamic visual stimuli using the Fove eye-tracking headset. It presents chromatic light stimuli (red and blue spheres) in various visual field positions while recording pupil data and event markers via the Lab Streaming Layer (LSL).

Problem Statement

This project enables controlled visual stimulation in virtual reality using the **Fove** headset and **LSL (Lab Streaming Layer)** to capture:

- **Visual Stimuli Events** – Red/Blue sphere presentations at different locations.
- **Eye Tracking Data** – Real-time pupil radius data from the Fove headset.

Useful for psychophysics experiments, visual neuroscience, and eye-tracking studies.

Goals

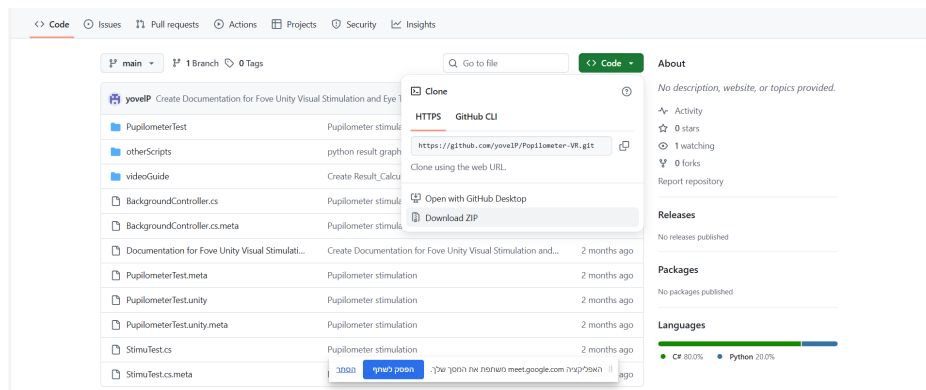
- Dynamic 3D visual stimuli (colored spheres).
- Eye position-dependent stimulus rendering.
- Pupil size collection using Fove SDK.
- Event and data streaming via LSL.
- Fixation light for gaze stability.
- Adjustable luminance and positions.Hypothesis

Scope

1. Set up LSL:
 1. Import the LSL C# bindings into the Unity project (liblsl-Csharp).
 2. Ensure LSL runtime is installed and available on your system path.
2. Set up Fove SDK:
 1. Install Fove Unity SDK.
 2. Add the FoveInterface prefab to your scene.

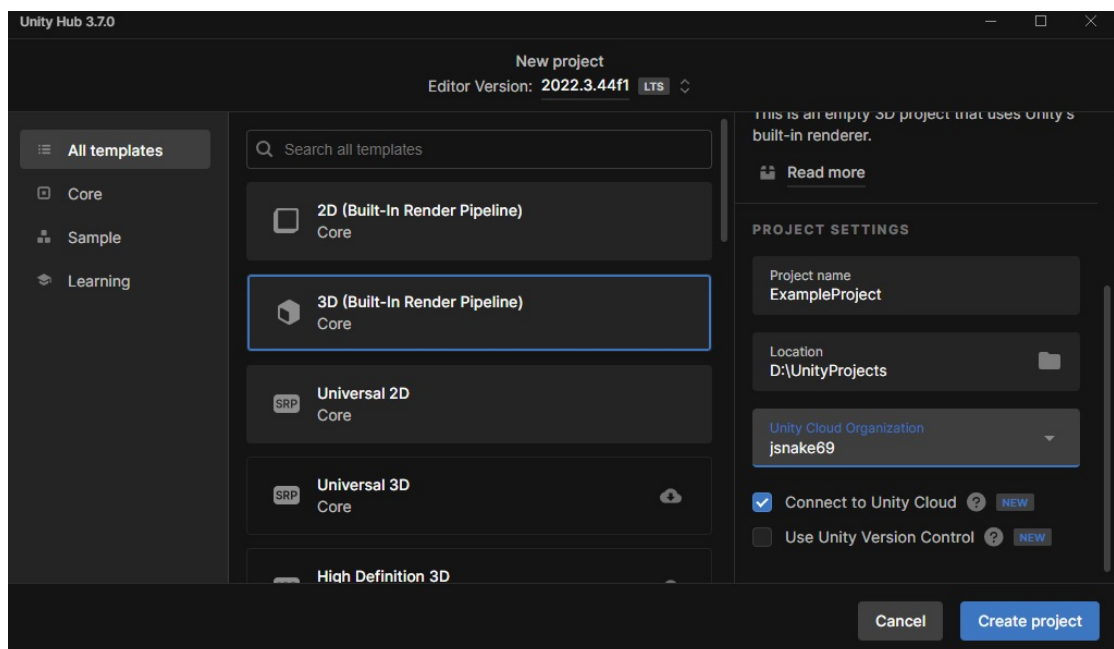
3. Set Unity project:

1. Go to [GitHub - yovelP/Popilometer-VR](#)
2. Click on "Code" and then on "Download Zip".



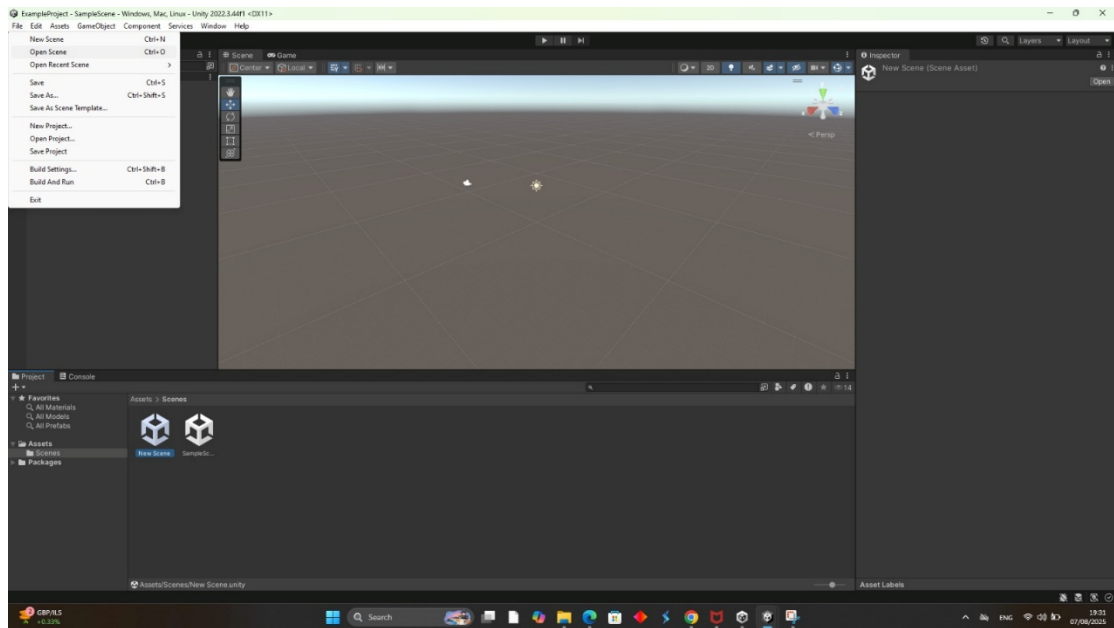
Option 1: Open the Scene pupilometerTest.unity:

1. Locate Your Project:
 - a. In Unity Hub, you'll see a list of your projects under the "Projects" tab.
 - b. If your project is not listed, click open (or add, depending on your Unity Hub version)
2. Configure the project name, location and unity cloud:

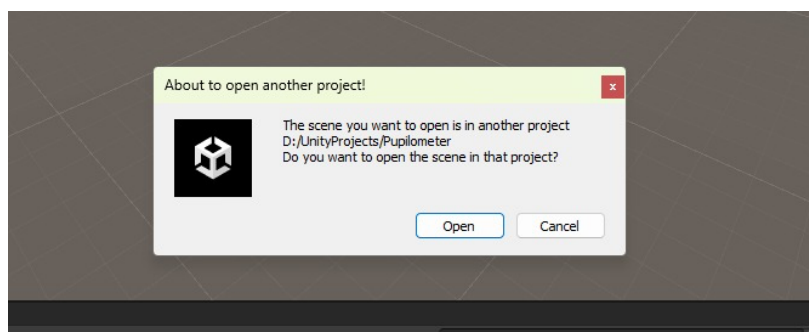
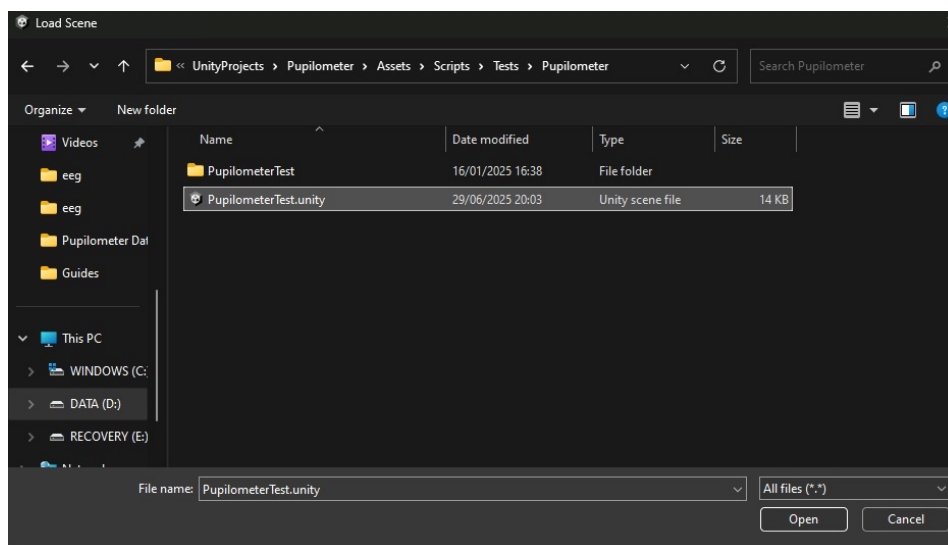


4. Open the Project – Click on your project's name in Unity Hub. Unity will now launch the Unity Editor and load your project.

5. Open the Scene – Click file -> open scene.



6. Navigate to the folder where your pupilometerTest.unity scene is saved.



7. Unity will now open that scene in the editor. Open the project in Unity.

Option 2: Create a new scene:

1. Scene Setup:
 1. Add the StimuTest and FoveBackgroundController scripts to an empty GameObject.
 2. Assign necessary parameters in the Inspector (or use defaults).
2. Play the scene in the Unity Editor.
3. After the initial delay (WaitForSec), stimulus presentation will begin.
4. Real-time pupil data and event markers will be sent via LSL.

Requirements:

- Unity 2020+ (recommended, we used 2022.3.44f1)
- [Fove Unity SDK](<https://developer.fove.io/>)
- [Lab Streaming Layer (LSL)](<https://github.com/scn/labstreaminglayer>) (LabRecorder.exe)
- Fove VR headset (with runtime installed) and updated license

Scripts Breakdown:

FoveBackgroundController.cs

Background Controller

- `foveCamera.backgroundColor = backgroundColor * luminance;`

Parameters (Editable in Inspector – under Fove Interface)

Parameter	Description	Default
backgroundColor	Background color	blue
Luminance	Light level	0.3

StimuTest.cs

Stimuli Presentation

- Red and blue spheres are presented at 5 default locations:
 - Center, Left, Right, Up, Down.
- Red/Blue spheres: 0.5s on, 3s off.
- Long blue: 8s on/off, repeated multiple times.

Fixation Light

- White sphere centered 8.5 units forward from Fove headset.

Eye Tracking

- Captures pupil radius from left and right eyes.
- Sends data every frame using LSL.StreamOutlet.

Event Markers

- Start/Stop events for stimuli are timestamped and streamed via LSL.

Parameters (Editable in Inspector – under Stimulation)

Parameter	Description	Default
stimulationDurationRedBlue	Duration (sec) red/blue spheres	0.5
intervalDurationRedBlue	Delay between red/blue spheres	3.0
stimulationDurationLongBlue	Long blue duration (sec)	8.0
intervalDurationLongBlue	Delay between long blues	8.0
focalBlueLightIterations	Repeats of long blue	3
WaitForSec	Initial wait before first stimulus	30
blueCircleSize	Blue circle size	0.1
redCircleSize	Red circle size	0.1

Parameter	Description	Default
blueLuminance	Blue luminance	0.3
redLuminance	Red luminance	0.3
LongBlueLuminance	Long Blue Luminance	6000.0
fixationLightSize	Fixation Light Size	0.09
fixationLightLuminance	Fixation Light Luminance	0.5
WaitForSec	Seconds to wait before starting the stimulation	30
vectorPositions	Default lights positions	(0,0,2),(-0.73,0,2), (0.73,-,2),(0,0.73,2), (0,-0.73,2)

LSL Streams

- Stream 1: UnityPupilData (float32[2])
 - Left and right pupil radius in mm.
- Stream 2: UnityEventStream (string)
 - Event markers: e.g., "Start Red sphere for 0.5 seconds in (0, 0, 2)"

Stimulation session setup instructions:

1. Open unity hab
2. Open pupilometer
3. Go to lab recorder under - C:\Users\GuestVR\Downloads\LabRecorder-1.16.4-Win_amd64\LabRecorder
4. Open LabRecorder.exe
5. Change study root to D:\Pupilometer Data
6. Update Participant number
7. Click start
8. Go to the unity project and press the start bottom
9. Go back to LabRecorder and press update
10. Mark all the streams
11. Stop the LabRecorder after the stimulation is over

Note: There is a recording with the instructions. You can find them locally under D:\Pupilometer Data\Guides\ Pupilometer_video_guide.mp4 or in the git repository.

Future Evolutions:

Real-Time Dashboard - Add a real-time UI showing:

- Current stimulus
- Eye status (validity)
- Retry in case of eye tracking failure
- Pupil size live values
- Control left or right eye
- Export eye's data as CSV file according to the current Shiba format

Stimulation session setup instructions:

This Python utility allows researchers to visualize and inspect pupil data and event markers recorded during a stimulation session. It loads XDF files recorded via LabRecorder, extracts relevant streams, and plots the left and right pupil size over time, with event markers clearly overlaid for reference.

Features

- Loads .xdf recording files using a file dialog.
- Extracts and plots left/right pupil size data (from UnityPupilData stream).
- Overlays event markers (from UnityEventStream) as vertical lines.
- Interactive hover tool shows marker labels (e.g., “Start Red sphere...”).
- Useful for data analysis.

Requirements:

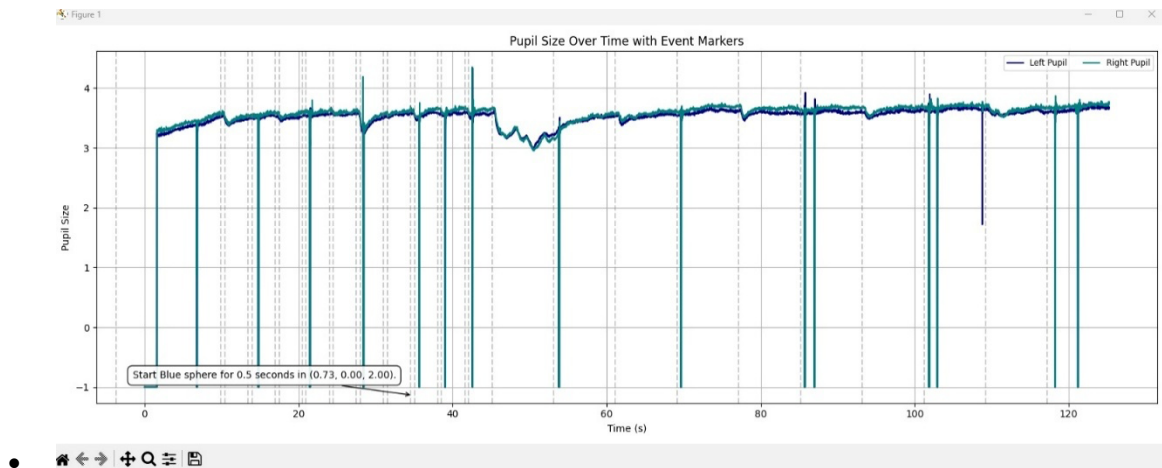
- Python 3.13
- pyxdf, numpy, matplotlib, mplcursors, tkinter (built-in)
- Install dependencies (if needed): "pip install pyxdf matplotlib mplcursors"

File Selection:

A file dialog will open upon script execution — select your .xdf file (e.g., from D:\Pupilometer Data).

Output:

- Interactive plot of pupil size over time.
- Vertical dashed lines for stimulus events.
- Cursor hover reveals the label of each event marker.



Example Use Cases:

- Verify pupil response according to different stimulations.
- Check data quality and stream presence.
- Confirm correct stimulus timing and recording start/stop points.

Usage Flow:

1. Run the script (python resultCalculator.py)
2. Select an .xdf file from the dialog
3. Plot will automatically render with pupil traces and events

Author

- Name: *Yovel Pando and Roni Sova*
- GitHub: <https://github.com/yovelP/Popilometer-VR>